ICPC Team Notebook

2026

Carlos E. R. Oliveira • Kaique A. de Oliveira • Mateus A. Mizael

Sumário

1 Matemática **2**
1.1 Combinatória . . . 2
1.1.1 Stars and Bars . . . 2
1.1.2 Princípio da Inclusão-Exclusão (PIE) . . . 2
1.1.3 Triângulo de Pascal . . . 2
1.1.4 Coeficientes Binomiais (Combinatória Otimizada) . . . 3
1.1.5 Permutação . . . 3
1.2 Fast Pow . . . 3
1.3 Inverso Modular . . . 3
1.4 Teorema Chinês dos Restos (CRT) . . . 4
1.5 Totient de Euler . . . 4
1.6 Linearidade da Expectativa e Variáveis Indicadoras . . . 5
1.7 Fast Fourier Transform (FFT) . . . 5
1.8 Exponenciação de Matrizes . . . 6
1.9 Miller-Rabin e Pollard Rho . . . 6
2 Geometria Computacional **7**
2.1 Primitivas Básicas . . . 7
2.2 Convex Hull (Monotone Chain) . . . 8
2.3 Rotating Calipers (Diâmetro do Polígono) . . . 8
2.4 Intersecção de Segmentos de Reta . . . 8
2.5 Intersecção de Retas . . . 9
2.6 Distância de Ponto a Reta e Segmento . . . 9
2.7 Área de Polígono (Shoelace) . . . 9
2.8 Ponto em Polígono (Ray Casting) . . . 9
3 Estruturas de Dados **10**
3.1 Fenwick Tree . . . 10
3.2 Merge Sort Tree . . . 10
3.3 Li Chao Tree . . . 11
3.4 Persistent Segment Tree . . . 11
4 Grafos **12**
4.1 Bellman-Ford . . . 12
4.2 Lowest Common Ancestor (Binary Lifting) . . . 12
4.3 Heavy-Light Decomposition (HLD) . . . 13
4.4 Centroid Decomposition . . . 13
4.5 Dinic's Algorithm (Max Flow) . . . 13
4.6 Pontes em Grafos (Tarjan) . . . 14
4.7 Hopcroft-Karp (Maximum Bipartite Matching) . . . 14
4.8 Min-Cost Max-Flow (MCMF) . . . 15
4.9 2-SAT Solver . . . 16
5 Programação Dinâmica **17**
5.1 Divide and Conquer DP Optimization . . . 17
5.2 SOS DP (Sum Over Subsets) . . . 17
5.3 Digit DP Template . . . 17
6 Strings **18**
6.1 KMP . . . 18
6.2 Z-Function . . . 18
6.3 Polynomial Rolling Hash . . . 19
6.4 Manacher's Algorithm . . . 19
6.5 Aho-Corasick Automaton . . . 20
6.6 Suffix Automaton (SAM) . . . 20
7 Misc **21**

7.1 Big Integer 21
7.2 Contagem de Inversões 23
7.3 Compressão de Coordenadas 23
7.4 Algoritmo de Mo (Range Queries Offline) 23
7.5 Meet-in-the-Middle 24

1. Matemática

1.1 Combinatória

Combinações distintas escolhendo k elementos dentre n elementos, sem importar a ordem:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Propriedade útil:

$$\binom{n}{k} = \binom{n}{n-k}$$

1.1.1 Stars and Bars

Utilizado para distribuir n objetos idênticos em k caixas distintas.

- **Caso 1 (Variáveis não-negativas, $x_i \geq 0$):** O número de soluções inteiras para a equação $x_1+x_2+\cdots+x_k = n$ é:
$$\binom{n+k-1}{k-1}$$
- **Caso 2 (Variáveis estritamente positivas, $x_i \geq 1$):** O número de soluções inteiras para a equação $x_1+x_2+\cdots+x_k = n$ é:
$$\binom{n-1}{k-1}$$

1.1.2 Princípio da Inclusão-Exclusão (PIE)

Calcula o tamanho da união de conjuntos ou conta elementos que evitam um conjunto de restrições.

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_i |A_i| - \sum_{i<j} |A_i \cap A_j| + \sum_{i<j<k} |A_i \cap A_j \cap A_k| - \cdots + (-1)^{n-1}|A_1 \cap \cdots \cap A_n|$$

Implementação clássica via bitmask (ideal para $n \leq 20$):

```
// Exemplo geral para contar elementos usando PIE via bitmask
ll total_union = 0;
int n_sets = 15; // numero de condicoes/conjuntos
for (int mask = 0; mask < (1 << n_sets); mask++) {
    ll size_intersection = get_intersection_size(mask);
    if (__builtin_popcount(mask) % 2 == 1) total_union += size_intersection;
    else total_union -= size_intersection;
} // Se o problema pede elementos que nao violam nenhuma condicao: Total - total_union
```

1.1.3 Triângulo de Pascal

Pré-computa combinações do tipo $\binom{n}{k}$. Seguro mesmo para módulos não primos.

Complexidade: Pré-computação em $O(n^2)$ e consultas em $O(1)$.

```
long long C[MAXN][MAXN];
void build_pascal() {
    for (int i = 0; i < MAXN; i++) {
        C[i][0] = 1; // nC0 = 1
        for (int j = 1; j <= i; j++) {
            C[i][j] = (C[i-1][j-1] + C[i-1][j]) % MOD;
        }
    }
}
```

1.1.4 Coeficientes Binomiais (Combinatória Otimizada)

Válido para MOD primo.

Complexidade: Pré-computação em $O(n \log(\text{MOD}))$ e consultas em $O(1)$.

```
ll fact[MAXN], invFact[MAXN];

void precompute() {
    fact[0] = invFact[0] = 1;
    for (int i = 1; i < MAXN; i++)
        fact[i] = (fact[i - 1] * i) % MOD;
    invFact[MAXN - 1] = modInverse(fact[MAXN - 1], MOD);
    for (int i = MAXN - 2; i >= 1; i--)
        invFact[i] = (invFact[i + 1] * (i + 1)) % MOD;
}
// iterativamente em O(r): res = res * (n - r + i) / i; i = [1, k]
ll nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
}
```

1.1.5 Permutação

Número de maneiras de escolher e ordenar k elementos dentre n:

$$P(n,k) = \frac{n!}{(n-k)!} = n(n-1)\cdots(n-k+1)$$

Complexidade: $O(k)$.

```
ll P(int n, int k) {
    ll res = 1;
    for (int i = 0; i < k; i++) res *= n - i;
    return res;
}
```

1.2 Fast Pow

Complexidade: $O(\log b)$.

```
ll binpow(ll a, ll b, ll mod) {
    a %= mod;
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
```

1.3 Inverso Modular

x é o inverso modular de a módulo m se:

$$ax \equiv 1 \pmod{m}$$

Existe se e somente se:

$$\gcd(a,m) = 1$$

Se m é primo:

$$a^{-1} \equiv a^{m-2} \pmod{m}$$

Complexidade: $O(\log m)$.

Requer: m primo e $a \not\equiv 0 \pmod{m}$.

```
ll modInverse(ll a, ll mod) {
    return binpow(a, mod - 2, mod);
}
```

1.4 Teorema Chinês dos Restos (CRT)

Resolve sistemas de congruências lineares: $x \equiv a_i \pmod{m_i}$ para $i = 0, \ldots, n-1$.

O algoritmo abaixo funciona mesmo se os módulos m_i **não forem coprimos**. Retorna $\{x, M\}$ onde a solução geral é $x \pmod{M}$ ou $\{-1, -1\}$ se não houver solução.

Complexidade: $O(n \log(\min(m)))$.

```
// Retorna o gcd e calcula x, y tais que a*x + b*y = gcd(a, b)
ll extgcd(ll a, ll b, ll &x, ll &y) {
    if (!b) { x = 1; y = 0; return a; }
    ll d = extgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}
// Retorna {x, M} tal que x == A[i] (mod M[i]) ou {-1, -1} se inconsistente
pair<ll, ll> crt(vector<ll> A, vector<ll> M) {
    ll a1 = A[0], m1 = M[0];
    for (size_t i = 1; i < A.size(); i++) {
        ll a2 = A[i], m2 = M[i];
        ll p, q;
        ll g = extgcd(m1, m2, p, q);
        if ((a2 - a1) % g != 0) return {-1, -1}; // Sem solucao

        __int128 mod = (__int128)m1 * (m2 / g);
        __int128 k = (a2 - a1) / g;
        __int128 m2_g = m2 / g;
        p = (p % m2_g + m2_g) % m2_g;
        __int128 diff = (__int128)p * (k % m2_g) % m2_g;

        __int128 x = (__int128)a1 + (__int128)m1 * diff;
        a1 = (x % mod + mod) % mod;
        m1 = mod;
    }
    return {a1, m1};
}
```

1.5 Totient de Euler

$\varphi(n)$ = quantidade de inteiros em $[1, n]$ coprimos com n.

Se:

$$n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$$

então:

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

Complexidade: $O(\sqrt{n})$.

```
ll phi(ll n) {
    ll res = n;
    for (ll p = 2; p * p <= n; p++) {
        if (n % p == 0) {
            while (n % p == 0) n /= p;
            res -= res / p;
        }
    }
    if (n > 1) res -= res / n;
    return res;
}
```

1.6 Linearidade da Expectativa e Variáveis Indicadoras

A esperança (valor esperado) da soma de variáveis aleatórias é sempre igual à soma das suas esperanças, **independentemente de haver dependência entre elas**:

$$E\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} E[X_i]$$

Para uma variável indicadora I_A de um evento A (onde $I_A = 1$ se A ocorre, 0 caso contrário):

$$E[I_A] = P(A)$$

1. Defina X como a variável total que você quer calcular o valor esperado.
2. Tente expressar $X = \sum_{i=1}^{n} X_i$, onde cada X_i é o menor subproblema/evento possível.
3. Transforme X_i em uma variável indicadora $I_i \in \{0, 1\}$.
4. Lembre-se: $E[X] = \sum E[I_i] = \sum P(\text{evento } i \text{ ocorrer})$.
5. Se os $P(\text{evento } i)$ forem idênticos por simetria, $E[X] = n \cdot P(\text{evento 1})$.

1.7 Fast Fourier Transform (FFT)

O tamanho do vetor resultante deve ser ajustado para a /textbfpróxima potência de 2.

Complexidade: $O(n \log n)$

```
using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> &a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i + j], v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= wlen;
            }
        }
    }
    if (invert) for (auto &x : a) x /= n;
}
vector<ll> multiply(vector<ll>& a, vector<ll>& b) {
    vector<cd> fa(all(a)), fb(all(b));
    int n = 1;
    while (n < a.size() + b.size()) n <<= 1;
    fa.resize(n); fb.resize(n);

    fft(fa, false); fft(fb, false);
    for (int i = 0; i < n; i++) fa[i] *= fb[i];
    fft(fa, true);

    vector<ll> result(n);
    for (int i = 0; i < n; i++) result[i] = round(fa[i].real());
    return result;
}
```

1.8 Exponenciação de Matrizes

Complexidade: $O(n^3 \log p)$

```
using Matrix = vector<vector<ll>>;

Matrix multiply(const Matrix &A, const Matrix &B) {
    int n = A.size();
    Matrix C(n, vector<ll>(n, 0));
    for (int i = 0; i < n; i++) {
        for (int k = 0; k < n; k++) {
            if (A[i][k] == 0) continue;
            for (int j = 0; j < n; j++)
                C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % MOD;
        }
    }
    return C;
}
Matrix power(Matrix A, ll p) {
    int n = A.size();
    Matrix res(n, vector<ll>(n, 0));
    for (int i = 0; i < n; i++) res[i][i] = 1; // Matriz Identidade
    while (p > 0) {
        if (p & 1) res = multiply(res, A);
        A = multiply(A, A);
        p >>= 1;
    }
    return res;
}
```

1.9 Miller-Rabin e Pollard Rho

pow_mod: $O(\log e)$

is_prime: $O(\log n)$

pollard_rho: $O(\sqrt[4]{n} \log n)$

factorize: $O(\sqrt[4]{n} \log n)$

```
using u64 = uint64_t;
using u128 = __uint128_t;
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

u64 mul_mod(u64 a, u64 b, u64 mod) {
    return (u128)a * b % mod;
}
u64 pow_mod(u64 a, u64 e, u64 mod) {
    u64 r = 1;
    while (e > 0) {
        if (e & 1) r = mul_mod(r, a, mod);
        a = mul_mod(a, a, mod);
        e >>= 1;
    }
    return r;
}
bool is_prime(u64 n) {
    if (n < 2) return false;
    for (u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29ULL, 31ULL, 37ULL}) {
        if (n % p == 0) return n == p;
    }
    u64 d = n - 1;
    int s = 0;
    while ((d & 1) == 0) {
        d >>= 1;
        ++s;
    }
    for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL}) {
        if (a % n == 0) continue;
        u64 x = pow_mod(a % n, d, n);
```

```
        if (x == 1 || x == n - 1) continue;
        bool witness = true;
        for (int r = 1; r < s; ++r) {
            x = mul_mod(x, x, n);
            if (x == n - 1) {
                witness = false;
                break;
            }
        }
        if (witness) return false;
    }
    return true;
}
u64 pollard_rho(u64 n) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    while (true) {
        u64 c = uniform_int_distribution<u64>(1, n - 1)(rng);
        u64 x = uniform_int_distribution<u64>(0, n - 1)(rng);
        u64 y = x;
        u64 d = 1;
        auto f = [&](u64 v) -> u64 {
            return ((u128)mul_mod(v, v, n) + c) % n;
        };
        while (d == 1) {
            x = f(x);
            y = f(f(y));
            u64 diff = (x > y ? x - y : y - x);
            d = __gcd(diff, n);
        }
        if (d != n) return d;
    }
}
void factorize(u64 n, map<u64, int> &factors) {
    if (n == 1) return;
    if (is_prime(n)) {
        factors[n]++;
        return;
    }
    u64 d = pollard_rho(n);
    factorize(d, factors);
    factorize(n / d, factors);
}
```

2. Geometria Computacional

2.1 Primitivas Básicas

```
const double EPS = 1e-9;
struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}

    Point operator+(const Point &p) const { return Point(x + p.x, y + p.y); }
    Point operator-(const Point &p) const { return Point(x - p.x, y - p.y); }
    Point operator*(double c) const { return Point(x * c, y * c); }
    Point operator/(double c) const { return Point(x / c, y / c); }
    double cross(const Point &p) const { return x * p.y - y * p.x; }
    double dot(const Point &p) const { return x * p.x + y * p.y; }
    double dist2() const { return x * x + y * y; }
    double dist(const Point &p) const { return sqrt((*this - p).dist2()); }
};
// Retorna o sentido do produto vetorial (> 0 se anti-horario, < 0 se horario, 0 se colinear)
double orientation(Point a, Point b, Point c) {
    return (b - a).cross(c - a);
}
```

2.2 Convex Hull (Monotone Chain)

Complexidade: $O(n \log n)$.

```
// Retorna os pontos do Convex Hull em sentido anti-horario
vector<Point> convex_hull(vector<Point> pts) {
    int n = pts.size(), k = 0;
    if (n <= 2) return pts;
    vector<Point> h(2 * n);
    sort(all(pts), [](Point a, Point b) {
        return a.x < b.x || (a.x == b.x && a.y < b.y);
    });
    for (int i = 0; i < n; i++) {
        while (k >= 2 && (h[k - 1] - h[k - 2]).cross(pts[i] - h[k - 2]) <= 0) k--;
        h[k++] = pts[i];
    }
    for (int i = n - 2, t = k + 1; i >= 0; i--) {
        while (k >= t && (h[k - 1] - h[k - 2]).cross(pts[i] - h[k - 2]) <= 0) k--;
        h[k++] = pts[i];
    }
    h.resize(k - 1);
    return h;
}
```

2.3 Rotating Calipers (Diâmetro do Polígono)

Complexidade: $O(n)$.

Requer: Os pontos devem formar um polígono convexo ordenado no sentido anti-horário (sem pontos colineares duplicados).

```
double rotating_calipers(const vector<Point>& hull) {
    int n = hull.size();
    if (n == 1) return 0;
    if (n == 2) return hull[0].dist2(hull[1]);

    double max_dist_sq = 0;
    int k = 1;
    while (abs((hull[1] - hull[0]).cross(hull[k + 1] - hull[0])) >
           abs((hull[1] - hull[0]).cross(hull[k] - hull[0]))) {
        k = (k + 1) % n;
    }

    int i = 0, j = k;
    while (i <= k && j < n) {
        max_dist_sq = max(max_dist_sq, (hull[i] - hull[j]).dist2());
        Point dir_i = hull[(i + 1) % n] - hull[i];
        Point dir_j = hull[(j + 1) % n] - hull[j];
        if (dir_i.cross(dir_j) > 0) j = (j + 1) % n;
        else i++;
    }
    return max_dist_sq; // retorna o quadrado da distancia
}
```

2.4 Intersecção de Segmentos de Reta

Complexidade: $O(1)$.

```
bool on_segment(Point a, Point b, Point c) {
    return c.x >= min(a.x, b.x) - EPS && c.x <= max(a.x, b.x) + EPS &&
           c.y >= min(a.y, b.y) - EPS && c.y <= max(a.y, b.y) + EPS;
}
bool intersect(Point a, Point b, Point c, Point d) {
    double o1 = orientation(a, b, c);
    double o2 = orientation(a, b, d);
    double o3 = orientation(c, d, a);
    double o4 = orientation(c, d, b);
    if (((o1 > EPS && o2 < -EPS) || (o1 < -EPS && o2 > EPS)) &&
        ((o3 > EPS && o4 < -EPS) || (o3 < -EPS && o4 > EPS))) {
```

```
        return true;
    }
    if (abs(o1) <= EPS && on_segment(a, b, c)) return true;
    if (abs(o2) <= EPS && on_segment(a, b, d)) return true;
    if (abs(o3) <= EPS && on_segment(c, d, a)) return true;
    if (abs(o4) <= EPS && on_segment(c, d, b)) return true;
    return false;
}
```

2.5 Intersecção de Retas

Complexidade: $O(1)$.

```
bool line_intersection(Point a, Point b, Point c, Point d, Point &out) {
    double denom = (b - a).cross(d - c);
    if (abs(denom) <= EPS) return false; // Retas paralelas ou coincidentes
    double t = (c - a).cross(d - c) / denom;
    out = a + (b - a) * t;
    return true;
}
```

2.6 Distância de Ponto a Reta e Segmento

Complexidade: $O(1)$.

```
double dist_point_to_line(Point p, Point a, Point b) {
    return abs((b - a).cross(p - a)) / a.dist(b);
}
double dist_point_to_segment(Point p, Point a, Point b) {
    Point ab = b - a;
    Point ap = p - a;
    Point bp = p - b;
    if (ap.dot(ab) <= 0.0) return p.dist(a);
    if (bp.dot(ab) >= 0.0) return p.dist(b);
    return abs(ab.cross(ap)) / a.dist(b);
}
```

2.7 Área de Polígono (Shoelace)

Requer vértices ordenados em sentido horário ou anti-horário.

Complexidade: $O(n)$.

```
double polygon_area_2(const vector<Point>& poly) {
    double area = 0;
    int n = poly.size();
    for (int i = 0; i < n; i++) {
        Point p1 = poly[i];
        Point p2 = poly[(i + 1) % n];
        area += p1.cross(p2);
    }
    return abs(area); // retorna o dobro da area
}
```

2.8 Ponto em Polígono (Ray Casting)

Complexidade: $O(n)$.

```
bool point_in_polygon(Point p, const vector<Point>& poly) {
    int n = poly.size();
    bool inside = false;
    for (int i = 0, j = n - 1; i < n; j = i++) {
        if ((poly[i].y > p.y) != (poly[j].y > p.y) &&
            p.x < (poly[j].x - poly[i].x) * (p.y - poly[i].y) / (poly[j].y - poly[i].y) + poly[i].x) {
            inside = !inside;
        }
```

```
    }
    return inside;
}
```

3. Estruturas de Dados

3.1 Fenwick Tree

Suporta soma de prefixo e atualização pontual.

Atualização: $O(\log n)$

Consulta de prefixo: $O(\log n)$

Soma em $[l, r]$**:** $O(\log n)$

Usa índices de 1 a n.

```
int n;
vector<ll> bit;
void init (int size) {
    n = size;
    bit.assign(n + 1, 0);
}
void add (int i, ll x) {
    for (; i <= n; i += i & -i) bit[i] += x;
}
ll sum (int i) {
    ll res = 0;
    for (; i > 0; i -= i & -i) res += bit[i];
    return res;
}
ll query (int l, int r) {
    return sum(r) - sum(l - 1);
}
```

3.2 Merge Sort Tree

Permite consultas de ordem em intervalos usando `lower_bound` / `upper_bound`.

Construção: $O(n \log n)$.

Consulta: $O(\log^2 n)$.

```
int n;
vector<vector<int>> tree;
void build (const vector<int>& a, int p, int l, int r) {
    if (l == r) {
        tree[p] = {a[l]};
        return;
    }
    int m = (l + r) / 2;
    build(a, p * 2, l, m);
    build(a, p * 2 + 1, m + 1, r);
    merge(tree[p * 2].begin(), tree[p * 2].end(),
          tree[p * 2 + 1].begin(), tree[p * 2 + 1].end(), back_inserter(tree[p]));
}
int query (int ql, int qr, int x, int p, int l, int r) {
    if (qr < l || r < ql) return 0;
    if (ql <= l && r <= qr)
        return lower_bound(tree[p].begin(), tree[p].end(), x) - tree[p].begin();
    int m = (l + r) / 2;
    return query(ql, qr, x, p * 2, l, m) + query(ql, qr, x, p * 2 + 1, m + 1, r);
}
```

O exemplo acima conta elementos $< x$ no intervalo $[l, r]$.

Para contar elementos $\leq x$, use `upper_bound`.

3.3 Li Chao Tree

Mantém um conjunto de retas $y = ax+b$ e responde a consultas de menor valor em um ponto x em $O(\log x)$, aceitando inserções arbitrárias.

Inserção: $O(\log x)$.

Consulta: $O(\log x)$.

```
struct Line {
    ll m, c;
    ll eval(ll x) { return m * x + c; }
};
struct LiChao {
    int sz;
    vector<Line> tree;
    LiChao(int n) {
        sz = n;
        tree.assign(4 * sz, {0, INF}); // Use -INF se for para maximo
    }
    void add_line(int node, int l, int r, Line new_line) {
        int mid = (l + r) / 2;
        bool better_mid = new_line.eval(mid) < tree[node].eval(mid);
        bool better_l = new_line.eval(l) < tree[node].eval(l);
        if (better_mid) swap(tree[node], new_line);
        if (l == r) return;
        if (better_l != better_mid) add_line(2 * node, l, mid, new_line);
        else add_line(2 * node + 1, mid + 1, r, new_line);
    }
    ll query(int node, int l, int r, ll x) {
        int mid = (l + r) / 2;
        ll res = tree[node].eval(x);
        if (l == r) return res;
        if (x <= mid) return min(res, query(2 * node, l, mid, x));
        else return min(res, query(2 * node + 1, mid + 1, r, x));
    }
};
```

3.4 Persistent Segment Tree

Mantém múltiplas versões de uma árvore de segmentos. Cada atualização cria apenas um novo caminho de nós ($O(\log n)$ novos nós por operação).

Complexidade: $O(\log n)$ em tempo e espaço por versão criada ou consulta.

```
const int MAXN = 2e5 + 5;
const int MAXNODES = MAXN * 40; // Ajuste conforme os limites de memoria
struct Node {
    int val;
    int left, right;
} pool[MAXNODES];

int root_ver[MAXN];
int node_cnt = 0;
int update(int prev_node, int l, int r, int idx, int val) {
    int cur_node = ++node_cnt;
    pool[cur_node] = pool[prev_node];
    pool[cur_node].val += val;
    if (l == r) return cur_node;
    int mid = (l + r) / 2;
    if (idx <= mid)
        pool[cur_node].left = update(pool[prev_node].left, l, mid, idx, val);
    else
        pool[cur_node].right = update(pool[prev_node].right, mid + 1, r, idx, val);
    return cur_node;
}
int query(int node, int l, int r, int ql, int qr) {
    if (!node || ql > r || qr < l) return 0;
    if (ql <= l && r <= qr) return pool[node].val;
    int mid = (l + r) / 2;
```

```
    return query(pool[node].left, l, mid, ql, qr) +
           query(pool[node].right, mid + 1, r, ql, qr);
}
```

4. Grafos

4.1 Bellman-Ford

Complexidade: $O(ve)$.

```
using Edge = tuple<ll, int, int>;
pair<vector<ll>, bool> bellmanFord (int n, const vector<Edge>& edges, int origin) {
    vector<ll> dist(n, INF);
    dist[origin] = 0;
    for (int i = 0; i < n - 1; i++) {
        bool changed = false;
        for (auto [u, v, w] : edges) {
            if (dist[u] == INF) continue;
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                changed = true;
            }
        }
        if (!changed) break;
    }
    for (const Edge& edge : edges) {
        if (dist[u] == INF) continue;
        if (dist[u] + w < dist[v]) return {dist, true};
    }
    return {dist, false};
}
```

4.2 Lowest Common Ancestor (Binary Lifting)

Pré-Processamento: $O(n \log n)$.

Consulta: $O(\log n)$.

```
const int MAXN = 2e5 + 5;
const int LOGN = 20;
vector<int> adj[MAXN];
int up[MAXN][LOGN];
int depth[MAXN];

void dfs_lca(int u, int p) {
    up[u][0] = p;
    for (int i = 1; i < LOGN; i++)
        up[u][i] = up[up[u][i - 1]][i - 1];
    for (int v : adj[u]) {
        if (v == p) continue;
            depth[v] = depth[u] + 1;
            dfs_lca(v, u);
    }
}
int lca(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);
    for (int i = LOGN - 1; i >= 0; i--) {
        if (depth[u] - (1 << i) >= depth[v])
            u = up[u][i];
    }
    if (u == v) return u;
    for (int i = LOGN - 1; i >= 0; i--) {
        if (up[u][i] != up[v][i]) {
            u = up[u][i];
            v = up[v][i];
        }
    }
    return up[u][0];
}
```

4.3 Heavy-Light Decomposition (HLD)

Complexidade: $O(\log^2 n)$.

```
vector<int> adj[MAXN];
int sz[MAXN], depth[MAXN], parent[MAXN], heavy[MAXN], head[MAXN], pos[MAXN];
int cur_pos;

void dfs_sz(int u, int p, int d) {
    sz[u] = 1; depth[u] = d; parent[u] = p;
    heavy[u] = -1;
    int max_sz = 0;
    for (int v : adj[u]) {
        if (v != p) {
            dfs_sz(v, u, d + 1);
            sz[u] += sz[v];
            if (sz[v] > max_sz) { max_sz = sz[v]; heavy[u] = v; }
        }
    }
}
void dfs_hld(int u, int h) {
    head[u] = h;
    pos[u] = ++cur_pos;
    if (heavy[u] != -1) dfs_hld(heavy[u], h);
    for (int v : adj[u])
        if (v != parent[u] && v != heavy[u]) dfs_hld(v, v);
}
```

4.4 Centroid Decomposition

Complexidade: $O(n \log n)$.

```
vector<set<int>> adj_c; // Grafo mutavel (arestas removidas)
vector<int> sz_c, parent_centroid;

void get_sz(int u, int p) {
    sz_c[u] = 1;
    for (int v : adj_c[u])
        if (v != p) { get_sz(v, u); sz_c[u] += sz_c[v]; }
}
int get_centroid(int u, int p, int total) {
    for (int v : adj_c[u])
        if (v != p && sz_c[v] > total / 2) return get_centroid(v, u, total);
    return u;
}
void decompose(int u, int p) {
    get_sz(u, p);
    int centroid = get_centroid(u, p, sz_c[u]);
    parent_centroid[centroid] = p;
    for (int v : adj_c[centroid]) {
        adj_c[v].erase(centroid);
        decompose(v, centroid);
    }
    adj_c[centroid].clear();
}
```

4.5 Dinic's Algorithm (Max Flow)

Complexidade: $O(v^2 e)$ (muito mais rápido na prática).

```
struct Edge { int to; ll cap, flow; int rev; };
struct Dinic {
    int n;
    vector<vector<Edge>> adj;
    vector<int> level, ptr;
    Dinic(int n) : n(n), adj(n), level(n), ptr(n) {}
    void add_edge(int from, int to, ll cap) {
        adj[from].push_back({to, cap, 0, (int)adj[to].size()});
```

```
        adj[to].push_back({from, 0, 0, (int)adj[from].size() - 1});
    }
    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        level[s] = 0; queue<int> q; q.push(s);
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (auto& edge : adj[v]) {
                if (edge.cap - edge.flow > 0 && level[edge.to] == -1) {
                    level[edge.to] = level[v] + 1; q.push(edge.to);
                }
            }
        }
        return level[t] != -1;
    }
    ll dfs(int v, int t, ll pushed) {
        if (pushed == 0 || v == t) return pushed;
        for (int& cid = ptr[v]; cid < adj[v].size(); cid++) {
            auto& edge = adj[v][cid];
            int tr = edge.to;
            if (level[v] + 1 != level[tr] || edge.cap - edge.flow == 0) continue;
            ll tr_pushed = dfs(tr, t, min(pushed, edge.cap - edge.flow));
            if (tr_pushed == 0) continue;
            edge.flow += tr_pushed;
            adj[tr][edge.rev].flow -= tr_pushed;
            return tr_pushed;
        }
        return 0;
    }
    ll max_flow(int s, int t) {
        ll flow = 0;
        while (bfs(s, t)) {
            fill(all(ptr), 0);
            while (ll pushed = dfs(s, t, 1e18)) flow += pushed;
        }
        return flow;
    }
};
```

4.6 Pontes em Grafos (Tarjan)

Complexidade: $O(n + m)$.

```
vector<int> adj_t[MAXN];
int tin[MAXN], low[MAXN], timer_t;
vector<pair<int, int>> bridges;

void dfs_bridge(int v, int p = -1) {
    tin[v] = low[v] = ++timer_t;
    for (int to : adj_t[v]) {
        if (to == p) continue;
        if (tin[to]) low[v] = min(low[v], tin[to]);
        else {
            dfs_bridge(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v]) bridges.push_back({min(v, to), max(v, to)});
        }
    }
}
```

4.7 Hopcroft-Karp (Maximum Bipartite Matching)

Complexidade: $O(e\sqrt{v})$.

```
struct HopcroftKarp {
    int n;
    vector<vector<int>> adj;
```

```
    vector<int> pair_u, pair_v, dist;
    HopcroftKarp(int n, int m) : n(n), adj(n + 1), pair_u(n + 1, 0), pair_v(m + 1, 0), dist(n + 1) {}
    void add_edge(int u, int v) { adj[u].push_back(v); }
    bool bfs() {
        queue<int> q;
        for (int u = 1; u <= n; u++) {
            if (pair_u[u] == 0) { dist[u] = 0; q.push(u); }
            else dist[u] = 1e9;
        }
        dist[0] = 1e9;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            if (u != 0) {
                for (int v : adj[u]) {
                    if (dist[pair_v[v]] == 1e9) {
                        dist[pair_v[v]] = dist[u] + 1; q.push(pair_v[v]);
                    }
                }
            }
        }
        return dist[0] != 1e9;
    }
    bool dfs(int u) {
        if (u != 0) {
            for (int v : adj[u]) {
                if (dist[pair_v[v]] == dist[u] + 1 && dfs(pair_v[v])) {
                    pair_v[v] = u; pair_u[u] = v; return true;
                }
            }
            dist[u] = 1e9; return false;
        }
        return true;
    }
    int max_matching() {
        int matching = 0;
        while (bfs()) {
            for (int u = 1; u <= n; u++) if (pair_u[u] == 0 && dfs(u)) matching++;
        }
        return matching;
    }
};
```

4.8 Min-Cost Max-Flow (MCMF)

Complexidade: $O(e\sqrt{v})$.

```
struct Edge {
    int to, cap, cost, flow, rev;
};
struct MCMF {
    int n;
    vector<vector<Edge>> adj;
    vector<ll> dist, potential;
    vector<int> parent_edge, parent_node;

    MCMF(int n) : n(n), adj(n), dist(n), potential(n, 0), parent_edge(n), parent_node(n) {}

    void add_edge(int from, int to, int cap, int cost) {
        adj[from].push_back({to, cap, cost, 0, (int)adj[to].size()});
        adj[to].push_back({from, 0, -cost, 0, (int)adj[to].size() - 1});
    }
    bool dijkstra(int s, int t) {
        fill(all(dist), 1e18);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
        dist[s] = 0;
        pq.push({0, s});

        while (!pq.empty()) {
```

```
            auto [d, u] = pq.top(); pq.pop();
            if (d > dist[u]) continue;
            for (int i = 0; i < adj[u].size(); i++) {
                auto& edge = adj[u][i];
                if (edge.cap - edge.flow > 0 && dist[edge.to] > dist[u] + edge.cost + potential[u] -
    potential[edge.to]) {
                    dist[edge.to] = dist[u] + edge.cost + potential[u] - potential[edge.to];
                    parent_node[edge.to] = u;
                    parent_edge[edge.to] = i;
                    pq.push({dist[edge.to], edge.to});
                }
            }
        }
        for (int i = 0; i < n; i++) {
            if (dist[i] < 1e18) potential[i] += dist[i];
        }
        return dist[t] < 1e18;
    }
    pair<ll, ll> min_cost_max_flow(int s, int t) {
        ll flow = 0, cost = 0;
        while (dijkstra(s, t)) {
            ll push = 1e18;
            for (int v = t; v != s; v = parent_node[v]) {
                auto& edge = adj[parent_node[v]][parent_edge[v]];
                push = min(push, (ll)(edge.cap - edge.flow));
            }
            for (int v = t; v != s; v = parent_node[v]) {
                auto& edge = adj[parent_node[v]][parent_edge[v]];
                edge.flow += push;
                adj[v][edge.rev].flow -= push;
                cost += push * edge.cost;
            }
            flow += push;
        }
        return {flow, cost};
    }
};
```

4.9 2-SAT Solver

Complexidade: $O(n + m)$.

```
struct TwoSAT {
    int n;
    vector<vector<int>> adj;
    vector<int> comp, tin, low, st;
    vector<bool> answer, on_stack;
    int timer, scc_cnt;

    TwoSAT(int n) : n(n), adj(2 * n), comp(2 * n, -1), tin(2 * n, 0), low(2 * n, 0), answer(n), on_stack(2 *
     n, false), timer(0), scc_cnt(0) {}

    void add_clause(int u, bool neg_u, int v, bool neg_v) {
        adj[2 * u + !neg_u].push_back(2 * v + neg_v);
        adj[2 * v + !neg_v].push_back(2 * u + neg_u);
    }
    void dfs(int u) {
        tin[u] = low[u] = ++timer;
        st.push_back(u); on_stack[u] = true;
        for (int v : adj[u]) {
            if (!tin[v]) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            }
            else if (on_stack[v])
                low[u] = min(low[u], tin[v]);
        }
        if (low[u] == tin[u]) {
```

```
            while (true) {
                int v = st.back(); st.pop_back();
                on_stack[v] = false;
                comp[v] = scc_cnt;
                if (u == v) break;
            }
            scc_cnt++;
        }
    }
    bool solve() {
        for (int i = 0; i < 2 * n; i++) if (!tin[i]) dfs(i);
        for (int i = 0; i < n; i++) {
            if (comp[2 * i] == comp[2 * i + 1]) return false;
            answer[i] = (comp[2 * i] > comp[2 * i + 1]);
        }
        return true;
    }
};
```

5. Programação Dinâmica

5.1 Divide and Conquer DP Optimization

Otimiza DPs da forma $dp[i][j] = \min_{k<j}(dp[i-1][k] + C(k,j))$ quando há monotonia no ponto ótimo ($opt(i,j) \leq opt(i,j+1)$).

Complexidade: $O(kn \log n)$.

```
ll dp_before[MAXN], dp_curr[MAXN];
ll cost(int i, int j) {
    // Retorna o custo de transicao do intervalo [i, j]
    return 0;
}
void compute(int l, int r, int opt_l, int opt_r) {
    if (l > r) return;
    int mid = (l + r) / 2;
    pair<ll, int> best = {1e18, -1};

    for (int k = opt_l; k <= min(mid, opt_r); k++)
        best = min(best, {dp_before[k] + cost(k + 1, mid), k});

    dp_curr[mid] = best.first;
    int opt = best.second;
    compute(l, mid - 1, opt_l, opt);
    compute(mid + 1, r, opt, opt_r);
}
```

5.2 SOS DP (Sum Over Subsets)

Calcula a soma sobre todos os subconjuntos de cada máscara de bits.

Complexidade: $O(n2^n)$.

```
// F[mask] armazena o valor inicial.
// Ao termino do loop, F[mask] contera a soma dos valores de todos os subconjuntos de mask.
void sos_dp(int n, vector<ll>& F) {
    for (int i = 0; i < n; ++i) {
        for (int mask = 0; mask < (1 << n); ++mask) {
            if (mask & (1 << i)) {
                F[mask] += F[mask ^ (1 << i)];
            }
        }
    }
}
```

5.3 Digit DP Template

Conta números em um intervalo $[0, n]$ que satisfazem certas propriedades baseadas em dígitos usando memoização.

```
string s;
ll memo[20][2]; // Ajuste os estados extras conforme o problema
ll dp(int idx, bool tight, /* outros estados necessarios */) {
    if (idx == s.size()) return 1; // Retorna 1 se encontrou um numero valido
    if (memo[idx][tight] != -1) return memo[idx][tight];

    ll ans = 0;
    int limit = tight ? (s[idx] - '0') : 9;

    for (int d = 0; d <= limit; ++d) {
        // Validacoes extras para o digito 'd' entram aqui se necessario
        ans += dp(idx + 1, tight && (d == limit) /*, novos estados */);
    }
    return memo[idx][tight] = ans;
}

ll solve(ll n) {
    if (n < 0) return 0;
    s = to_string(n);
    memset(memo, -1, sizeof(memo));
    return dp(0, true /*, estado inicial */);
}
// Para resolver [A, B], faca solve(B) - solve(A - 1)
```

6. Strings

6.1 KMP

Prefix Function: $O(m)$.

Busca: $O(n)$.

```
vector<int> prefixFunction (const string& pattern) {
    int pLen = pattern.size();
    vector<int> pref(pLen);
    for (int i = 1; i < pLen; i++) {
        int j = pref[i - 1];
        while (j > 0 && pattern[i] != pattern[j])
            j = pref[j - 1];
        if (pattern[i] == pattern[j]) j++;
        pref[i] = j;
    }
    return pref;
}
int kmp (const string& text, const string& pattern) {
    vector<int> pref = prefFunction(pattern);
    int pLen = pattern.size();
    for (int i = 0, j = 0; i < text.size(); i++) {
        while (j > 0 && text[i] != pattern[j])
            j = pref[j - 1];
        if (text[i] == pattern[j]) j++;
        if (j == pLen) return i - pLen + 1;
    }
    return -1;
}
```

6.2 Z-Function

Complexidade: $O(n)$.

```
vector<int> zFunction (const string& text) {
    int n = text.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && text[z[i]] == text[i + z[i]]) z[i]++;
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
```

```
    }
    return z;
}
```

6.3 Polynomial Rolling Hash

Utiliza double hashing para evitar colisões.

Complexidade: $O(n)$ para Pré-processamento.

```
struct DoubleHash {
    ll h1, h2;
    bool operator==(const DoubleHash& other) const {
        return h1 == other.h1 && h2 == other.h2;
    }
    bool operator<(const DoubleHash& other) const {
        if (h1 != other.h1) return h1 < other.h1;
        return h2 < other.h2;
    }
};
struct RollingHash {
    ll MOD1 = 1e9 + 7, MOD2 = 1e9 + 9;
    ll P1 = 31, P2 = 37;
    vector<ll> h1, h2, p1, p2;

    RollingHash(const string& s) {
        int n = s.size();
        h1.assign(n + 1, 0); h2.assign(n + 1, 0);
        p1.assign(n + 1, 1); p2.assign(n + 1, 1);
        for (int i = 0; i < n; i++) {
            p1[i+1] = (p1[i] * P1) % MOD1;
            p2[i+1] = (p2[i] * P2) % MOD2;
            h1[i+1] = (h1[i] * P1 + (s[i] - 'a' + 1)) % MOD1;
            h2[i+1] = (h2[i] * P2 + (s[i] - 'a' + 1)) % MOD2;
        }
    }
    DoubleHash get(int l, int r) { // 0-indexed, inclusive
        ll hash1 = (h1[r+1] - h1[l] * p1[r - l + 1]) % MOD1;
        if (hash1 < 0) hash1 += MOD1;
        ll hash2 = (h2[r+1] - h2[l] * p2[r - l + 1]) % MOD2;
        if (hash2 < 0) hash2 += MOD2;
        return {hash1, hash2};
    }
};
```

6.4 Manacher's Algorithm

Encontra o maior palíndromo centrado em cada posição (ímpar e par) de uma string.

Complexidade: $O(n)$.

```
vector<vector<int>> manacher(string s) {
    int n = s.size();
    vector<vector<int>> d(2, vector<int>(n, 0));
    for (int z = 0; z < 2; z++) {
        for (int i = 0, l = 0, r = -1; i < n; i++) {
            int k = (i > r) ? z : min(d[z][l + r - i + z], r - i + 1);
            while (0 <= i - k - z && i + k < n && s[i - k - z] == s[i + k]) k++;
            d[z][i] = k--;
            if (i + k > r) {
                l = i - k - z;
                r = i + k;
            }
        }
    }
    return d;
}
```

6.5 Aho-Corasick Automaton

Complexidade: $O(n + m + \text{matches})$.

```
const int MAXC = 26;
struct AhoCorasick {
    struct Node {
        int next[MAXC];
        int fail = 0;
        int elink = 0;
        vector<int> pids;
        Node() { memset(next, -1, sizeof(next)); }
    };

    vector<Node> trie;
    AhoCorasick() {
        trie.emplace_back();
    }
    void insert(const string& s, int id) {
        int u = 0;
        for (char c : s) {
            int idx = c - 'a';
            if (trie[u].next[idx] == -1) {
                trie[u].next[idx] = trie.size();
                trie.emplace_back();
            }
            u = trie[u].next[idx];
        }
        trie[u].pids.pb(id);
    }
    void build() {
        queue<int> q;
        for (int c = 0; c < MAXC; ++c) {
            if (trie[0].next[c] != -1) q.push(trie[0].next[c]);
            else trie[0].next[c] = 0;
        }
        while (sz(q)) {
            int u = q.front(); q.pop();
            for (int c = 0; c < MAXC; ++c) {
                int v = trie[u].next[c];
                if (v != -1) {
                    trie[v].fail = trie[trie[u].fail].next[c];
                    int f = trie[v].fail;
                    if (!trie[f].pids.empty()) trie[v].elink = f;
                    else trie[v].elink = trie[f].elink;
                    q.push(v);
                }
                else trie[u].next[c] = trie[trie[u].fail].next[c];
            }
        }
    }
    void search(const string& s, vector<vector<int>>& occur) {
        int n = sz(s);
        int u = 0;
        for (int i = 0; i < n; i++) {
            int idx = s[i] - 'a';
            u = trie[u].next[idx];
            int curr = u;
            while (curr > 0) {
                for (int id : trie[curr].pids) occur[id].pb(i);
                curr = trie[curr].elink;
            }
        }
    }
};
```

6.6 Suffix Automaton (SAM)

Complexidade: $O(n)$.

```
struct State {
    int len, link;
    int next[26]; // padrao para letras de 'a' a 'z', reajuste se necessario
};

const int MAXLEN = 1e5 + 5; // ajuste conforme o tamanho maximo esperado
State st[MAXLEN * 2];
int sz, last;

void sam_init() {
    st[0].len = 0;
    st[0].link = -1;
    memset(st[0].next, -1, sizeof(st[0].next));
    sz = 1;
    last = 0;
}
void sam_extend(char c) {
    int id = c - 'a'; // intervalo [0, 25], reajuste se necessario
    int cur = sz++;
    st[cur].len = st[last].len + 1;
    memset(st[cur].next, -1, sizeof(st[cur].next));

    int p = last;
    while (p != -1 && st[p].next[id] == -1) {
        st[p].next[id] = cur;
        p = st[p].link;
    }

    if (p == -1)
        st[cur].link = 0;
    else {
        int q = st[p].next[id];
        if (st[p].len + 1 == st[q].len) {
            st[cur].link = q;
        } else {
            int clone = sz++;
            st[clone].len = st[p].len + 1;
            memcpy(st[clone].next, st[q].next, sizeof(st[q].next));
            st[clone].link = st[q].link;
            while (p != -1 && st[p].next[id] == q) {
                st[p].next[id] = clone;
                p = st[p].link;
            }
            st[q].link = st[cur].link = clone;
        }
    }
    last = cur;
}
```

7. Misc

7.1 Big Integer

Utilitários

```
string stripLeadingZeros (const string &number) {
    int firstNonZero = number.find_first_not_of('0');
    if (firstNonZero == string::npos) return "0";
    return number.substr(firstNonZero);
}
```

Soma

Assume que ambos os números são positivos.

Complexidade: $O(n)$.

```
string addBig (const string &first, const string &second) {
```

```
    string result;
    int carry = 0;
    int i = first.size() - 1;
    int j = second.size() - 1;
    while (i >= 0 || j >= 0 || carry) {
        int digitSum = carry;
        if (i >= 0) digitSum += first[i--] - '0';
        if (j >= 0) digitSum += second[j--] - '0';
        result.push_back('0' + digitSum % 10);
        carry = digitSum / 10;
    }
    reverse(all(result));
    return stripLeadingZeros(result);
}
```

Subtração

Assume que $first \geq second$.

Complexidade: $O(n)$.

```
string subtractBig (const string &first, const string &second) {
    string result;
    int borrow = 0;
    int i = first.size() - 1;
    int j = second.size() - 1;
    while (i >= 0) {
        int digit = (first[i] - '0') - borrow;
        if (j >= 0)
            digit -= second[j--] - '0';
        if (digit < 0) {
            digit += 10;
            borrow = 1;
        } else {
            borrow = 0;
        }
        result.push_back('0' + digit);
        i--;
    }
    reverse(all(result));
    return stripLeadingZeros(result);
}
```

Multiplicação por Inteiro

Complexidade: $O(n)$.

```
string multiplyByInt (const string &number, int multiplier) {
    string result;
    ll carry = 0;
    for (int i = number.size() - 1; i >= 0; i--) {
        ll product = (number[i] - '0') * 1LL * multiplier;
        product += carry;
        result.push_back('0' + product % 10);
        carry = product / 10;
    }
    while (carry > 0) {
        result.push_back('0' + carry % 10);
        carry /= 10;
    }
    reverse(all(result));
    return stripLeadingZeros(result);
}
```

Divisão por Inteiro

Complexidade: $O(n)$.

```
pair<string, int> divideByInt (const string &number, int divisor) {
    string quotient;
    ll remainder = 0;
    for (char digit : number) {
        remainder = remainder * 10 + (digit - '0');
        quotient.push_back('0' + remainder / divisor);
        remainder %= divisor;
    }
    return {stripLeadingZeros(quotient), remainder};
}
```

Módulo por Inteiro

Complexidade: $O(n)$.

```
int modString (const string &number, int mod) {
    ll remainder = 0;
    for (char digit : number)
        remainder = (remainder * 10 + (digit - '0')) % mod;
    return remainder;
}
```

7.2 Contagem de Inversões

Complexidade: $O(n \log n)$.

```
ll count_inversions(vector<int>& a) {
    // Compressao de Coordenadas
    vector<int> vals = a;
    sort(all(vals));
    vals.erase(unique(all(vals)), vals.end());
    SegTree st(sz(vals)); // Segtree para soma e update = 1
    ll inv = 0;

    for (int x : a) {
        int rank = lower_bound(all(vals), x) - vals.begin();
        inv += st.query(rank + 1, vals.size() - 1);
        st.update(rank, 1);
    }
    return inv;
}
```

7.3 Compressão de Coordenadas

Mapeia valores grandes para um intervalo compacto $[1, k]$ preservando a ordem relativa em $O(n \log n)$.

Complexidade: $O(n + m + \text{matches})$.

```
vector<int> vals = a; // vetor original
sort(all(vals));
vals.erase(unique(all(vals)), vals.end());

auto get_id = [&](int x) {
    return lower_bound(all(vals), x) - vals.begin() + 1; // 1-indexed
};
// Para cada elemento a[i], use get_id(a[i])
```

7.4 Algoritmo de Mo (Range Queries Offline)

Complexidade: $O((n + q)\sqrt{n})$.

```
const int BLOCK = 450; // ~ sqrt(N)
struct Query {
    int l, r, id;
    bool operator<(const Query& other) const {
        int b1 = l / BLOCK, b2 = other.l / BLOCK;
```

```
        if (b1 != b2) return b1 < b2;
        return (b1 & 1) ? r < other.r : r > other.r; // Otimizacao de zigue-zague
    }
};
void add(int idx) { /* Adiciona o elemento na estrutura de dados */ }
void remove(int idx) { /* Remove o elemento da estrutura de dados */ }
long long get_answer() { /* Retorna a resposta atual */ return 0; }
vector<ll solve_mo(vector<Query>& queries, const vector<int>& a) {
    int q = queries.size();
    vector<ll ans(q);
    sort(queries.begin(), queries.end());
    int cur_l = 0, cur_r = -1;
    for (auto& qry : queries) {
        while (cur_l > qry.l) add(--cur_l);
        while (cur_r < qry.r) add(++cur_r);
        while (cur_l < qry.l) remove(cur_l++);
        while (cur_r > qry.r) remove(cur_r--);
        ans[qry.id] = get_answer();
    }
    return ans;
}
```

7.5 Meet-in-the-Middle

Complexidade: $O(2^{n/2} \log(2^{n/2}))$.

```
// Exemplo generico para subconjuntos (Subset Sum / Knapsack)
vector<ll generate_sums(const vector<ll& v) {
    int n = v.size();
    vector<ll sums;
    for (int mask = 0; mask < (1 << n); ++mask) {
        long long s = 0;
        for (int i = 0; i < n; ++i) {
            if (mask & (1 << i)) s += v[i];
        }
        sums.push_back(s);
    }
    sort(all(sums));
    return sums;
}
```